Mini Project using JAVA Programming

Enhancement Report

Project Title:

AutoConnect – ISP Automation System

A Console-Based System for Efficient Management of
Internet Service Provider Operations Using Java and OOP

Submitted By:
Pranjal Rajesh Upadhyay
PRN: 1032233275
Roll No: 39
Panel: B

Under the guidance of:
Prof. Suhas Joshi

1. Introduction

The AutoConnect ISP Automation System is a terminal-based Java application that manages ISP operations using OOP. The old version (v1.0) had a single flat menu for basic customer management, plan assignment, billing, and complaints. The new version (v2.0) introduces major enhancements including role-based access, authentication, password security, technician assignment, and admin controls.

2. List of Enhancements

1. Dual-Role Entry Point (Admin / Customer)
2. Admin Authentication with hardcoded credentials
3. Customer Registration with Email and Password
4. Password Strength Checker (PasswordUtil class)
5. Admin Dashboard with 8 options
6. Technician Assignment and Complaint Status Tracking
7. Admin: Suspend / Activate Customer toggle
8. Customer: Change Password
9. Customer-Specific Complaint View
10. OOP Model Simplification

3. Detailed Enhancements

3.1 Dual-Role Entry Point (Admin / Customer)

The old version had a single flat menu accessible to everyone without any role check. The new version shows a role-selection screen first, routing users to either Admin or Customer flow.

Old Code:

```
System.out.println("===== AutoConnect ISP System =====");
System.out.println("1. Register Customer");
System.out.println("2. View Customers");
int choice = sc.nextInt();
switch (choice) { case 1 -> { /* register */ } ... }
```

New Code:

```
System.out.println("1. Admin");
System.out.println("2. Customer");
System.out.println("3. Exit");
int role = sc.nextInt();
if (role == 1) { // Admin login + dashboard }
else if (role == 2) { // Customer register/login }
```

Change in Output:

- System now asks: "Who are you? 1.Admin 2.Customer 3.Exit"

- Unauthorized users cannot see admin options.

3.2 Admin Authentication with Hardcoded Credentials

No admin concept existed in the old version. The new version adds an Admin class and two pre-seeded admins. Login is verified via email + password. Wrong credentials show a security warning.

Old Code:

```
// No Admin class. No login. Anyone could access everything.
```

New Code:

```
class Admin {
    String name, email, password;
}

List<Admin> admins = new ArrayList<>(Arrays.asList(
    new Admin("Rajesh",  "rajeshu0707@gmail.com", "admin@01"),
    new Admin("Santosh", "santoshu555@gmail.com",  "admin@02")
));

Admin adminLogin(String email, String pass) {
    for (Admin a : admins)
        if (a.email.equals(email) && a.password.equals(pass)) return a;
    return null;
}
```

Change in Output:

- Failed login: "SECURITY THREAT DETECTED! Unauthorized access attempt. Access Denied."
- Successful login: "Welcome Admin Rajesh!"

3.3 Customer Registration with Email and Password

Old registration only collected Name and Phone. The new version adds Email and Password, and customers must log in before accessing features.

Old Code:

```
public void registerCustomer(String name, String phone) {
    customers.add(new Customer(customerIdCounter++, name, phone));
}

public Customer(int id, String name, String phone) { ... }
```

New Code:

```
Customer(int id, String n, String ph, String e, String pass) {
    this.id=id; name=n; phone=ph; email=e; password=pass;
}

Customer login(String email, String pass) {
    for (Customer c : customers)
        if (c.email.equals(email) && c.password.equals(pass)) return c;
```

```
    return null;
}
```

Change in Output:

- Registration now asks: Name, Phone, Email, Password.
- Invalid login shows: "Invalid credentials! Please register first."

3.4 Password Strength Checker – PasswordUtil Class

A new PasswordUtil class was added. It checks 5 criteria (length >= 8, uppercase, lowercase, digit, special character) and rates passwords as WEAK, MEDIUM, or STRONG. WEAK passwords are rejected during registration.

Old Code:

```
// No PasswordUtil class. No password field in Customer.
```

New Code:

```
class PasswordUtil {
    static String strength(String p) {
        int score = 0;
        if (p.length() >= 8)               score++;
        if (p.matches(".*[A-Z].*"))         score++;
        if (p.matches(".*[a-z].*"))         score++;
        if (p.matches(".*[0-9].*"))         score++;
        if (p.matches(".*[^a-zA-Z0-9].*")) score++;
        if (score <= 2) return "WEAK";
        else if (score <= 4) return "MEDIUM";
        else return "STRONG";
    }
    static boolean valid(String p) {
        return strength(p).equals("STRONG");
    }
}
```

Change in Output:

- WEAK passwords are rejected with a tip message during registration.
- MEDIUM passwords prompt: Use this or enter a stronger one?
- STRONG passwords are accepted and a Confirm Password step follows.

3.5 Admin Dashboard (8 Options)

No admin dashboard existed before. The new version gives a logged-in admin 8 exclusive options.

Old Code:

```
// No admin section. All operations were in one flat menu.
```

New Code:

```
System.out.println("===== Admin Dashboard =====");
System.out.println("1. View Yourself");
```

```
System.out.println("2. View All Customers");
System.out.println("3. View All Bills");
System.out.println("4. View All Complaints");
System.out.println("5. Assign Technician");
System.out.println("6. Update Complaint Status");
System.out.println("7. Suspend / Activate Customer");
System.out.println("8. Logout");
```

Change in Output:

- Admin can view all customers, bills, and complaints system-wide.
- Admin can assign technicians and update complaint status.
- Admin can toggle customer status (ACTIVE / SUSPENDED).

3.6 Technician Assignment and Complaint Status Tracking

The Complaint class gained a technician field. Two new methods were added: assignTechnician() and updateComplaintStatus(). This tracks who is handling a complaint and its current progress.

Old Code – Complaint class:

```
class Complaint {
    private int id, customerId;
    private String issue, status;
    // No technician field
    // status only toggled to "RESOLVED" via resolve()
}
```

New Code – Complaint class + service methods:

```
class Complaint {
    int id, cid; String issue;
    String status = "OPEN";
    String technician = "NOT ASSIGNED";   // NEW FIELD
}

void assignTechnician(int compId, String tech) {
    c.technician = tech;
    c.status = "IN PROGRESS";
}

void updateComplaintStatus(int compId, int choice) {
    // 1=RESOLVED | 2=IN PROGRESS | 3=WAITING
}
```

Change in Output:

- Complaints now show: ID | Issue | Status | Technician.
- Status changes to "IN PROGRESS" automatically when a technician is assigned.

3.7 Admin: Suspend / Activate Customer Toggle

The old code had a suspend() method on Customer but it was never wired to the UI. The new version exposes this as an admin toggle: ACTIVE becomes SUSPENDED and vice versa.

Old Code:

```
public void suspend() {
    this.status = ConnectionStatus.SUSPENDED;
}
// This method was never called from the menu.
```

New Code:

```
void toggleCustomerStatus(int customerId) {
    for (Customer c : customers) {
        if (c.id == customerId) {
            if (c.status == ConnectionStatus.ACTIVE) {
                c.status = ConnectionStatus.SUSPENDED;
            } else {
                c.status = ConnectionStatus.ACTIVE;
            }
        }
    }
}
```

Change in Output:

- "Customer 'Aisha' has been SUSPENDED." / "...ACTIVATED."

3.8 Customer: Change Password

A new option in the customer menu allows a logged-in customer to change their password. Only STRONG passwords are accepted.

Old Code:

```
// No password field. No changePassword() method.
```

New Code:

```
void changePassword(Customer c, String newPass) {
    c.password = newPass;
}

// In customer menu (case 10):
System.out.print("New Password: ");
String np = sc.next();
if (PasswordUtil.valid(np)) {
    s.changePassword(c, np);
    System.out.println("Password Changed Successfully!");
} else {
    System.out.println("Weak password! Not updated.");
}
```

Change in Output:

- Weak/medium passwords are rejected: "Weak password! Not updated."

3.9 Customer-Specific Complaint View

Old viewComplaints() showed ALL complaints to everyone. The new version filters by the logged-in customer's ID.

Old Code:

```
public void viewComplaints() {
    complaints.forEach(System.out::println);
}
```

New Code:

```
void viewComplaints(int cid) {
    boolean found = false;
    for (Complaint c : comps)
        if (c.cid == cid) {
            System.out.println("ID: "+c.id+" | Issue: "+c.issue
                +" | Status: "+c.status+" | Technician: "+c.technician);
            found = true;
        }
    if (!found) System.out.println("No complaints found.");
}
```

Change in Output:

- Each customer sees only their own complaints.
- Admin's viewAllComplaints() still shows all.

3.10 OOP Model Simplification

The old version used private fields with getters/setters. The new version uses package-private (default) fields for direct access, making the code shorter.

Old Code:

```
class Customer {
    private int id;
    private String name;
    private Plan plan;
    private ConnectionStatus status;
    public int getId() { return id; }
    public String getName() { return name; }
    // ... more getters/setters
}
```

New Code:

```
class Customer {
    int id;
    String name, phone, email, password;
    ConnectionStatus status = ConnectionStatus.SUSPENDED;
    // No getters/setters - direct field access
}
```

Note:

This reduces boilerplate but sacrifices strict OOP encapsulation. Acceptable for a single-package console project.

4. Sample Input / Output – Password Strength (New Version)

Name	Password	Strength	Outcome
Aisha	Aisha@1234	STRONG	Registered Successfully
Rahul	Rahul123	MEDIUM	Registered (user chose to keep)
Karan	karan123	WEAK	Rejected – asked to re-enter
Neha	neha	WEAK	Rejected – asked to re-enter
Arjun	Arjun@ML2025	STRONG	Registered Successfully

Console Output Sample:

```
Who are you? 1.Admin  2.Customer  3.Exit
> 2
1.Register  2.Login  3.Back
> 1
Name: Karan
Phone: 9012345678
Email: karan.tech@gmail.com
Password: karan123
Password Strength: WEAK
Password is too weak! Please enter a stronger password.
Tip: Use uppercase, lowercase, number & special character (min 8 chars).
Password: Karan@Tech9
Password Strength: STRONG
Confirm Password: Karan@Tech9
Registered Successfully!
```

5. Additional Suggestions / Possible Future Enhancements

1. Input Validation: Phone and email format should be validated using regex to avoid invalid data entry.
2. Data Persistence: All data is lost on exit. File I/O or a database (SQLite via JDBC) can be used for permanent storage.
3. Bill Amount from Plan: In the new version, generateBill() hardcodes Rs. 799. It should use the assigned plan's actual price.
4. Restore Encapsulation: Model classes now use public fields. Private fields with getters/setters should be restored for proper OOP.
5. Duplicate Email Check: Registration does not check for existing emails. Two accounts with the same email can cause login issues.
6. Bill History per Customer: A per-customer bill view should be added in the customer menu for better usability.
7. Plan Speed Info Removed: The Plan class had a speedMbps field in v1.0 which was removed in v2.0. Restoring it helps in plan comparison.

6. Conclusion

The new version of AutoConnect ISP Automation System (v2.0) is a significant improvement over v1.0. The addition of role-based access, email authentication, password strength validation, admin dashboard, technician assignment, and filtered complaint view makes the system more realistic and functional. The code remains fully console-driven with no external dependencies. Future work should focus on data persistence, input validation, and restoring strict OOP encapsulation to make the system production-ready.